

Lab Assignment ⁴
Object-Oriented Programming

Branch
B.E., 2nd Year – ENC



Submitted By:
Kaushtubh Kumar
Roll No. 1024150191
Batch: 2023

Question 1

Create a class named 'Rectangle' with two data members- length and breadth and a function to calculate the area which is 'length*breadth'. The class has three constructors: (1) No parameter - assign zero, (2) Two parameters, (3) One parameter - both get same value. Create objects with none, one and two parameters and print their areas.

Code

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length;
    double breadth;
public:
    Rectangle() {
        length = 0;
        breadth = 0;
    }
    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }
    Rectangle(double side) {
        length = breadth = side;
    }
    double area() {
        return length * breadth;
    }
    void display() {
        cout << "L=" << length << ", B=" << breadth << endl;
    }
};

int main() {
    Rectangle r1;
    r1.display();
    cout << "Area=" << r1.area() << endl;

    Rectangle r2(5);
    r2.display();
    cout << "Area=" << r2.area() << endl;

    Rectangle r3(4, 6);
    r3.display();
    cout << "Area=" << r3.area() << endl;

    return 0;
}
```

```
L=0, B=0
```

```
Area=0
```

```
L=5, B=5
```

```
Area=25
```

```
L=4, B=6
```

```
Area=24
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```

Question 2

Redefine the program with array of objects of Rectangle class, calculate area for each object. Implement constructors with default arguments and destructor.

Code

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length, breadth;
public:
    Rectangle(double l=0, double b=0) {
        length = l;
        breadth = b;
        cout << "Constructor" << endl;
    }
    ~Rectangle() {
        cout << "Destructor" << endl;
    }
    double area() {
        return length * breadth;
    }
    void display() {
        cout << "L=" << length
            << ", B=" << breadth
            << ", Area=" << area() << endl;
    }
};

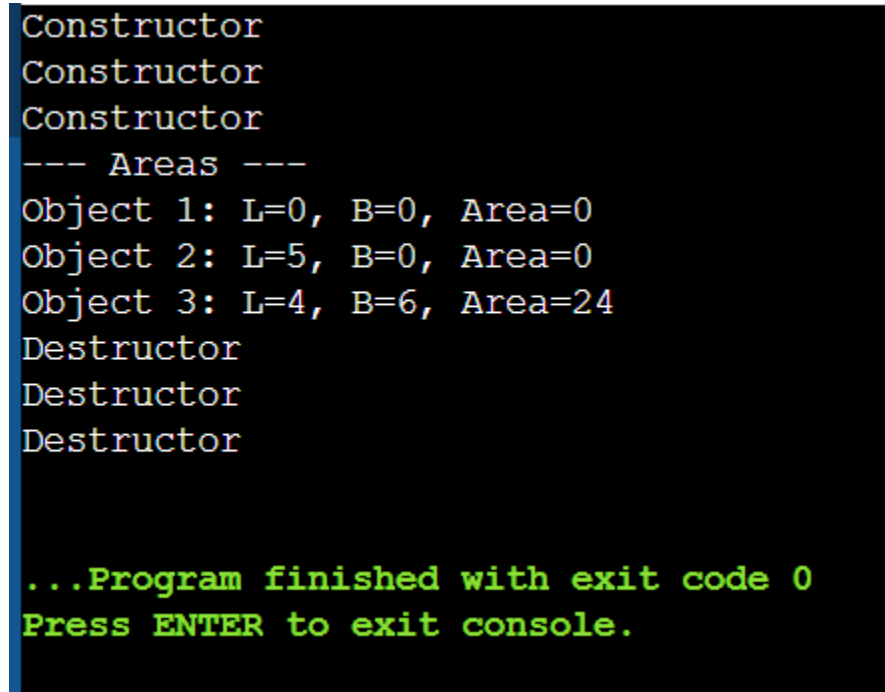
int main() {
    Rectangle arr[3] = {
        Rectangle(),
        Rectangle(5),
        Rectangle(5),
    };
}
```

```

        Rectangle(4, 6)
    };

    cout << "
--- Areas ---" << endl;
    for (int i = 0; i < 3; i++) {
        cout << "Object " << i+1 << ": ";
        arr[i].display();
    }
    return 0;
}

```



```

Constructor
Constructor
Constructor
--- Areas ---
Object 1: L=0, B=0, Area=0
Object 2: L=5, B=0, Area=0
Object 3: L=4, B=6, Area=24
Destructor
Destructor
Destructor

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3

Verify destructor properties: (1) Name begins with ~ and matches class name, (2) Only one destructor allowed, (3) No parameters, (4) No return type, (5) Compiler generates default when not specified.

Code

```

#include <iostream>
using namespace std;

class TestDestructor {
public:
    TestDestructor() {
        cout << "Constructor" << endl;
    }
    ~TestDestructor() {
        cout << "Destructor called" << endl;
    }
}

```

```

};

class NoDestructor {
    // No destructor defined
    // Compiler generates default
};

int main() {
    cout << "=== Properties 1, 3, 4 ===" << endl;
    {
        TestDestructor obj;
    }

    cout << "
=== Property 5 ===" << endl;
    {
        NoDestructor obj2;
        cout << "Default destructor used" << endl;
    }

    cout << "
=== Property 2 ===" << endl;
    cout << "Multiple destructors cause compile error" << endl;

    return 0;
}

```

```

=== Properties 1, 3, 4 ===
Constructor
Destructor called
=== Property 5 ===
Default destructor used
=== Property 2 ===
Multiple destructors cause compile error

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 4

Implement dynamic memory allocation using new and delete for: integer variable, float variable, integer array, float array, class objects, array of objects.

Code

```

#include <iostream>
using namespace std;

```

```

class Demo {
public:
    int value;
    Demo(int v) {
        value = v;
        cout << "Created: " << value << endl;
    }
    ~Demo() {
        cout << "Destroyed: " << value << endl;
    }
};

int main() {
    // Integer variable
    int* ip = new int;
    *ip = 42;
    cout << "int: " << *ip << endl;
    delete ip;

    // Float variable
    float* fp = new float;
    *fp = 3.14f;
    cout << "float: " << *fp << endl;
    delete fp;

    // Integer array
    int* ia = new int[5]{10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        cout << ia[i] << " ";
    cout << endl;
    delete[] ia;

    // Float array
    float* fa = new float[4]{1.5f, 2.5f, 3.5f, 4.5f};
    for (int i = 0; i < 4; i++)
        cout << fa[i] << " ";
    cout << endl;
    delete[] fa;

    // Class object
    Demo* obj = new Demo(100);
    delete obj;

    // Array of objects
    Demo* arr = new Demo[3];
    arr[0] = Demo(1);
    arr[1] = Demo(2);
    arr[2] = Demo(3);
    delete[] arr;

    cout << "All memory freed!" << endl;
    return 0;
}

```

```
Destroyed: 100  
Created default: 0  
Created default: 0  
Created default: 0  
Created: 1  
Destroyed: 1  
Created: 2  
Destroyed: 2  
Created: 3  
Destroyed: 3  
Destroyed: 3  
Destroyed: 2  
Destroyed: 1  
All memory freed!
```

```
...Program finished with exit code 0  
Press ENTER to exit console.
```